

FilersHub

Development Environment Setup Guide

Prepared for: Antigravity Development Team | March 2026

Overview

This document outlines how to set up a development/staging environment for FilersHub so new features and changes can be tested before deploying to production. The goal is a **two-branch workflow** where all development happens in a separate Vercel deployment while sharing the same Supabase database as production.

Current Stack

Service	Role
GitHub	Source control and version management
Supabase	Database, auth, edge functions, webhooks
Vercel	Frontend hosting and deployments
GoHighLevel	CRM platform (test location available with test data)
Claude Code	AI-assisted development (installed in Antigravity workspace)

Architecture: Two-Branch Setup

The setup uses two Git branches that each deploy to their own Vercel environment, while both connect to the **same Supabase project** (production database). This is intentional so development can be tested against real data structures and the existing test GHL location.

Branch Structure

Branch	Vercel URL	Purpose
main	app.filershub.com	Production. What all users see. Only receives merges from dev after testing is complete.
dev	dev.filershub.com	Development/staging. All new work happens here. Irene reviews on this URL before approving merge to main.

Setup Steps

Step 1: Create the dev Branch in GitHub

1. Create a new branch called **dev** from **main**

2. All feature branches should be created from and merged back into **dev**
3. Only merge **dev** into **main** after Irene has reviewed and approved on the dev URL

Step 2: Configure Vercel for Two Deployments

Vercel automatically creates preview deployments for non-production branches. To give the dev branch a stable URL:

4. In the Vercel project dashboard, go to Settings > Domains
5. Add a domain like **dev.filershub.com** (or use the auto-generated preview URL)
6. Assign that domain to the **dev** branch via the Git Branch dropdown
7. Production domain (app.filershub.com) stays assigned to **main**

Step 3: Environment Variables

Both branches will use the same Supabase credentials. In Vercel:

- Supabase URL, anon key, and service role key: **Same for both environments** (we are intentionally sharing the production database)
- GHL OAuth credentials: Use the existing app credentials. The test GHL location already has test data for validation.
- If any env vars need to differ between dev and production (e.g., webhook URLs, callback URLs), set them per-branch in Vercel under Settings > Environment Variables using the branch scope.

IMPORTANT: Shared Database

Since both environments share the same Supabase database, be cautious with any database migrations or schema changes on the dev branch. Coordinate before running migrations so production is not affected.

Step 4: GHL Webhook & Callback URLs

Any GHL workflow actions, triggers, or OAuth callbacks that reference a FilersHub URL will need attention:

- Production webhooks should continue pointing to **app.filershub.com**
- For testing in the dev GHL location, point webhooks to **dev.filershub.com**
- OAuth redirect URIs may need both domains whitelisted in the GHL marketplace app settings

Daily Development Workflow

For New Features

8. Create a feature branch from **dev** (e.g., feature/irs-alerts)
9. Build and test locally (Claude Code can be used for development here)
10. Push to GitHub and create a PR into **dev**
11. Vercel automatically creates a preview deployment for the PR
12. Merge PR into **dev** → deploys to dev.filershub.com
13. Irene reviews on the dev URL using the test GHL location

14. Once approved, merge **dev** into **main** → deploys to app.filershub.com (production)

For Hotfixes

Critical production bugs can be fixed directly on a branch from **main**, merged to **main**, then back-merged into **dev** to keep branches in sync.

Claude Code Integration

Claude Code is already installed in the Antigravity workspace. It fits into this workflow naturally:

- Use Claude Code on feature branches or the **dev** branch for writing, reviewing, and debugging code
- Claude Code can assist with Supabase migrations, edge function development, and Vercel configuration
- Since it runs in your terminal, it operates on whatever branch you have checked out — just make sure you are on **dev** (not **main**) when building new features
- No special configuration is needed — it works the same as before, just be branch-aware

Testing with the GHL Test Location

Irene has a dedicated test GHL location with sample data. When testing on the dev environment:

- Use the test location for all feature validation — do not test against client locations
- Test workflows, webhook triggers, and status changes in the test location
- Verify Chrome extension behavior against the dev URL
- Confirm client portal renders correctly on the dev deployment

Summary

Component	What Changes
GitHub	Add dev branch. All work goes to dev first.
Vercel	Assign dev.filershub.com to dev branch. Production stays on main.
Supabase	No changes. Both environments share the same project and database.
GHL	Test location points webhooks to dev URL for testing.
Claude Code	No changes. Just ensure you are on the correct branch.

Questions? Reach out to Irene directly.